Hair Oil E-Commerce

Admin: Add Products & Place Images Anywhere

Full file structure — Frontend (React) + Backend (Flask)

What's new in this structure

Feature	Description
Admin add/edit products	Full product form with images, variants, pricing, SEO, status
Media manager	Upload, browse, delete images across all folders
MediaPickerModal	Pick any uploaded image from any form field
Page builder	Drag blocks onto any page, place images anywhere
Block types	Hero, Banner, ImageText, ImageGrid, ProductRow, FAQ
JSONB page storage	Entire page layout stored as JSON in PostgreSQL
product_images table	Many-to-many: products ↔ media with sort order
Public page API	Frontend fetches page layout and renders each block

FRONTEND — Admin folder (new & updated)

```
dir admin/
├── AdminLayout.jsx   ← sidebar + topbar shell
├── AdminRoute.jsx    ← blocks non-admin users
│
├── pages/
│   ├── DashboardPage.jsx
│   ├── ProductsPage.jsx    ← list all products, search, filter
│   ├── AddProductPage.jsx  ← NEW PRODUCT FORM
│   ├── EditProductPage.jsx  ← EDIT PRODUCT FORM
│   ├── OrdersPage.jsx
│   ├── OrderDetailPage.jsx
│   ├── UsersPage.jsx
│   ├── CategoriesPage.jsx
│   └── MediaManagerPage.jsx   ← browse & manage all uploaded images
│
├── components/
│   ├── product/   ← product form pieces
│   │   ├── ProductForm.jsx   ← master form wrapper
│   │   ├── ProductBasicFields.jsx   ← name, slug, brand, price, stock
│   │   ├── ProductDescriptionEditor.jsx   ← rich text / textarea
│   │   ├── ProductCategorySelector.jsx   ← dropdown from DB
│   │   ├── ProductVariantManager.jsx   ← colors, sizes, variants
│   │   ├── ProductPricingFields.jsx   ← price, compare price, sale toggle
│   │   ├── ProductSEOFields.jsx   ← meta title, description, slug
│   │   ├── ProductStatusToggle.jsx   ← active / draft / soldout
│   │   └── ProductImageAttacher.jsx   ← pick from media library or upload
│   │
│   ├── media/   ← image management
│   │   ├── MediaLibrary.jsx   ← grid of all uploaded images
│   │   ├── MediaUploader.jsx   ← drag-and-drop + click to upload
│   │   ├── MediaImageCard.jsx   ← thumbnail + name + copy URL + delete
│   │   ├── MediaFolderTabs.jsx   ← switch: products/banners/pages/categories
│   │   └── MediaPickerModal.jsx   ← popup: pick image for any field
│   │
│   ├── page-builder/   ← place images anywhere on site
│   │   ├── PageBuilderEditor.jsx   ← main drag-and-drop canvas
│   │   ├── SectionList.jsx   ← list of page sections (hero, banner...)
│   │   ├── SectionBlock.jsx   ← individual editable section
│   │   ├── BlockToolbar.jsx   ← move up/down, delete, edit block
│   │   ├── blocks/
│   │   │   ├── HeroBlock.jsx   ← full-width hero: title + image + CTA
│   │   │   ├── BannerBlock.jsx   ← promo banner with image
│   │   │   ├── ImageTextBlock.jsx   ← image + text side by side
│   │   │   ├── ImageGridBlock.jsx   ← 2/3/4 col image grid
│   │   │   ├── ProductRowBlock.jsx   ← manual product selection row
│   │   │   ├── TextBlock.jsx   ← heading + paragraph
│   │   │   └── FAQBlock.jsx   ← accordion with image
│   │   └── PageBuilderPreview.jsx   ← live preview of page layout
│   │
│   └── shared/
│       ├── AdminSidebar.jsx
│       ├── StatsCard.jsx
│       ├── OrdersTable.jsx
│       ├── UsersTable.jsx
│       └── OrderStatusBadge.jsx
```

BACKEND — Admin routes + new DB migrations

```
dir app/admin/
├── __init__.py
├── dashboard.py   ← stats: orders, users, revenue, products
│
├── products.py   ← PRODUCT CRUD
    # Routes inside products.py:
       GET     /admin/products   ← list all (paginated, search)
       POST    /admin/products   ← create new product
       GET     /admin/products/<id>   ← get single product
       PUT     /admin/products/<id>   ← update product
       DELETE  /admin/products/<id>   ← delete product
       PUT     /admin/products/<id>/status   ← toggle active/draft/soldout
       POST    /admin/products/<id>/images   ← attach images to product
       DELETE  /admin/products/<id>/images/<img_id>   ← remove image from product
│
├── media.py   ← IMAGE MANAGER
    # Routes inside media.py:
       POST    /admin/media/upload   ← upload 1+ images (multipart)
       GET     /admin/media   ← list all files (folder filter)
       DELETE  /admin/media/<filename>   ← delete an uploaded file
       GET     /admin/media/folders   ← list folder names
│
├── page_builder.py   ← PAGE BUILDER / PLACE IMAGES ANYWHERE
    # Routes inside page_builder.py:
       GET     /admin/pages   ← list all custom pages
       POST    /admin/pages   ← create new page
       GET     /admin/pages/<slug>   ← get page with all blocks
       PUT     /admin/pages/<slug>   ← save full page (blocks JSON)
       DELETE  /admin/pages/<slug>   ← delete page
       GET     /pages/<slug>   ← PUBLIC: serve page to frontend
│
├── categories.py
├── orders.py
├── users.py
└── refunds.py
```

Uploads folder

```
dir uploads/   ← Flask serves this as /static/uploads/
    ├── products/   ← product images (named by product_id)
    ├── banners/   ← promo + hero banners
    ├── categories/   ← category card images
    └── pages/   ← images placed via page builder
```

Migrations (additions)

```
dir migrations/
    ├── 001_users.sql
    ├── 002_products.sql
    ├── 003_categories.sql
    ├── 004_cart.sql
    ├── 005_orders.sql
    ├── 006_wishlist.sql
    ├── 007_media.sql   ← image metadata: id, filename, folder, url, size, created_at
    ├── 008_product_images.sql   ← product_id FK → media_id FK (many-to-many)
    └── 009_pages.sql   ← custom pages: id, slug, title, blocks JSONB, updated_at
```

Add / Edit Product — File Reference

File / Folder	Purpose
`AddProductPage.jsx`	Full page: renders ProductForm in 'create' mode. On submit calls POST /admin/products
`EditProductPage.jsx`	Full page: loads product by ID, renders ProductForm in 'edit' mode. Calls PUT /admin/products/<id>
`ProductForm.jsx`	Master form wrapper. Holds all form state (useState). Splits into tab sections: Basic, Images, Pricing, SEO, Status
`ProductBasicFields.jsx`	Inputs: product name, brand, slug (auto-generated from name), short description, stock quantity
`ProductImageAttacher.jsx`	Shows attached images + button to open MediaPickerModal. Drag to reorder. First image = main/thumbnail
`ProductPricingFields.jsx`	Price, Compare-at price (shows sale badge), Cost price (hidden from customers)
`ProductVariantManager.jsx`	Add color/size variants. Each variant has its own price, stock, and image
`ProductStatusToggle.jsx`	Radio: Active (visible in store) / Draft (hidden) / Sold Out (shown with badge)
`ProductSEOFields.jsx`	Meta title, meta description, URL slug — for search engine visibility
`ProductCategorySelector.jsx`	Multi-select dropdown. Categories fetched from GET /admin/categories

Media Manager — File Reference

File / Folder	Purpose
`MediaManagerPage.jsx`	Admin page at /admin/media. Shows MediaFolderTabs + MediaLibrary grid. Allows bulk upload and delete
`MediaLibrary.jsx`	Responsive image grid. Each card shows thumbnail, filename, size. Click to copy URL or delete
`MediaUploader.jsx`	Drop zone + browse button. Accepts jpg/png/webp. Sends multipart POST /admin/media/upload. Shows progress
`MediaImageCard.jsx`	Thumbnail card with: preview, filename, folder tag, copy-URL button, delete button
`MediaFolderTabs.jsx`	Tabs: All / Products / Banners / Pages / Categories. Filters the MediaLibrary by folder param
`MediaPickerModal.jsx`	Popup used by ProductImageAttacher and page builder blocks. Search + browse library, click to select

Page Builder — place images anywhere

File / Folder	Purpose
`MediaManagerPage.jsx`	Admin creates pages at /admin/pages. Drag blocks to build layout. Each block has an image picker
`PageBuilderEditor.jsx`	Main canvas. Left: block palette. Center: live drag-and-drop canvas. Right: selected block settings panel
`SectionBlock.jsx`	Renders one block on canvas. Has toolbar: move up, move down, duplicate, delete, edit settings
`HeroBlock.jsx`	Settings: background image (MediaPicker), heading text, subtext, CTA button label + link
`BannerBlock.jsx`	Settings: background/foreground image, overlay text, badge text (e.g. 'UP TO 60% OFF'), button
`ImageTextBlock.jsx`	Settings: image (left or right), heading, paragraph, CTA. Used for 'Your Hair, Reimagined' sections
`ImageGridBlock.jsx`	Settings: 2/3/4 columns, each cell has its own image picker + optional caption/link
`ProductRowBlock.jsx`	Admin manually picks products to feature. Renders them as a horizontal product card row
`FAQBlock.jsx`	Settings: left image (MediaPicker) + list of Q&A pairs. Renders accordion FAQ with side image
`PageBuilderPreview.jsx`	Iframe/panel showing live render of current block layout as it would appear to customers

Backend Files — Product & Image APIs

File / Folder	Purpose
`admin/products.py`	Full CRUD for products. POST validates required fields, generates slug, saves to DB, returns new product JSON
`admin/media.py`	POST /upload: saves file to uploads/<folder>/, stores metadata in media table, returns {id, url, filename}
`admin/page_builder.py`	GET/PUT /admin/pages/<slug>: stores entire page layout as JSONB column in pages table. Public GET serves it
`migrations/007_media.sql`	Table: id, original_name, stored_name, folder, url, mime_type, size_bytes, uploaded_by, created_at
`migrations/008_product_images.sql`	Junction table: product_id (FK), media_id (FK), sort_order, is_primary. Allows multiple images per product
`migrations/009_pages.sql`	Table: id, slug (unique), title, blocks (JSONB), is_published, created_by, updated_at
`routes/uploads.py`	Public route: POST /upload (used during product add). Calls same logic as admin/media.py upload handler

Flow: How admin adds a new product

Step	Action
Step 1	Admin goes to /admin/products → clicks 'Add New Product'
Step 2	AddProductPage.jsx renders ProductForm.jsx with empty state
Step 3	Admin fills: name (slug auto-generated), brand, category, price, stock, description
Step 4	In the Images tab: clicks 'Add Images' → MediaPickerModal opens
Step 5	Modal shows MediaLibrary — admin can pick existing OR upload new via MediaUploader
Step 6	Selected images attach to ProductImageAttacher — admin drags to reorder, marks primary
Step 7	Admin sets Status: Active / Draft / Sold Out, fills SEO fields
Step 8	Clicks Save → ProductForm calls POST /admin/products with full JSON body
Step 9	Backend: validates, inserts into products table, inserts rows into product_images table
Step 10	Redirect to /admin/products — new product appears in list immediately

Flow: How admin places an image anywhere on the page

Step	Action
Step 1	Admin goes to /admin/pages → selects a page (e.g. Home) or creates new
Step 2	PageBuilderEditor.jsx loads current page blocks from GET /admin/pages/<slug>
Step 3	Admin drags a block from palette: Hero, Banner, ImageText, ImageGrid, etc.
Step 4	Block appears on canvas with placeholder. Admin clicks block to open settings panel
Step 5	Settings panel has an 'Image' field → clicking opens MediaPickerModal
Step 6	Admin browses MediaLibrary (all folders) or uploads a new image on the spot
Step 7	Selected image URL fills into the block's image field. Canvas updates live
Step 8	Admin can add multiple blocks, reorder by dragging, preview with PageBuilderPreview
Step 9	Clicks Publish → PUT /admin/pages/<slug> saves entire blocks array as JSONB
Step 10	Frontend fetches GET /pages/<slug> and renders each block using its type + data

New database tables summary

Table	Key columns	Purpose
media	id, filename, folder, url, size_bytes	Tracks every uploaded image file
product_images	product_id FK, media_id FK, sort_order, is_primary	Links products to their images (many-to-many)
pages	id, slug, title, blocks JSONB, is_published	Stores page builder layout as JSON
products (updated)	Added: status ENUM, meta_title, meta_desc	Extended for SEO + draft/active/soldout